SYS3 LAB ACTIVITY 3 : Register Renaming

Weeks 8 (typical study period)

Introduction:

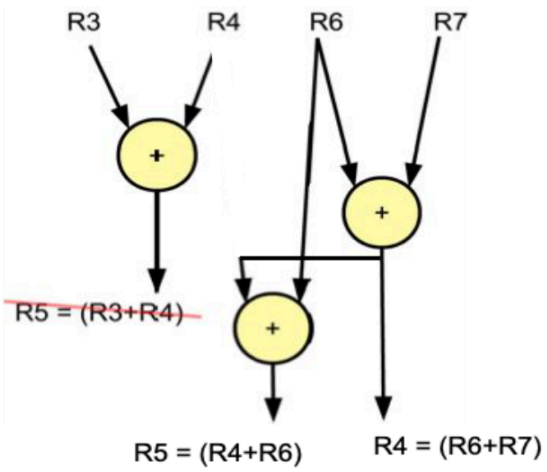
As mentioned previously, register hazards occur when register read and write sequences have the potential to get out of order and create problems. Remember the three key register hazard definitions:-

Read After write, RAW: An instruction needs to read a register that is written previously, but in the pipeline the write is delayed for some reason.

Write After Read WAR: An Instruction writes to a register after it is read, but due to the read being delayed, it ends up writing to the register *before* it has been read.

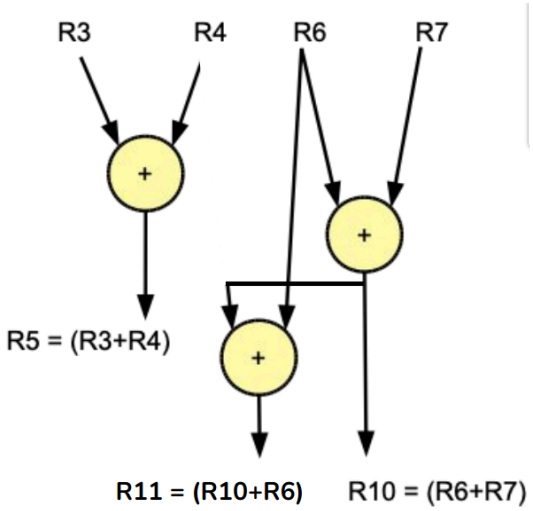
Write after Write, WAW: An instruction overwrites a previous write to the same register, but due to the order being incorrect the new value is overwritten by the old value instead.

In Activity 3 we attempted to solve these issue by inserting delays so that the offending register references are separated in the time domain. However there is an alternative for some cases, known as register renaming. Consider the following example:

i1 IADD R3,R4,R5 **i2 IADD R7,R6,R4** **i3 IADD R4,R6,R5**	 **Data Flow Diagram**

We can see several dependencies here. First of all i2 overwrites **R4**, which i1 wants to read. This is a *potential* Write after Read (WAR) hazard, meanwhile i3 wants to read **R4** which is being written by i2, a potential RAW hazard. Secondly, i3 writes to register **R5** as does i1. This is a potential WAW hazard. These cases could be a problem if the pipeline sequence becomes delayed for some reason (and with a more mixed combination of instructions this will also be observed more often).

Now Consider a revised example:-

i1 IADD R3,R4,R5 **i2 IADD R7,R6,R10** **i3 IADD R10,R6,R11**	 **Data Flow Diagram**

Now i2 writes to a new register **R10**, and no-longer clashes with R4 in i1. Likewise, i3 writes to a new register R11 and no longer clashes with R5 in i1. Assuming that lots of registers are available, we can therefore eliminate any number of WAR and WAW hazards without adding stalls. This means code has fewer delays, and therefore it executes faster[1].

An important consideration is that once a register is renamed, then all following references to that register must also be renamed. So every reference to register R4 from i2 onward must be renamed as R10, and every reference to R5 from i3 onward must also be renamed. If this is done, then the data-flow will be preserved. The **data flow diagrams** given in the example confirm that the code performs the same computations, only the register numbers are changed.

One problem that renaming cannot resolve is RAW hazards. This is a true dependency, where a read cannot proceed until a preceding write has completed. Renaming the registers will not help here. Therefore we can say that renaming will remove WAR and WAW hazards, but RAW hazards can only be fixed by inserting stalls.

In this practical we will attempt to develop solutions that use register renaming to resolve WAR and WAW hazards.

[1] The downside of this is that by using more registers we create a need for a larger register file, and this is a significant hardware cost for silicon area and power consumption in any modern CPU. Advanced static algorithms can attempt to work out when a particular register name is no longer needed (a concept related to 'register lifetime'), and thus when it is safe to reuse the register 'name' without creating a hazard. This allows a smaller register file to support the same optimisations. Another alternative is to use a virtual register file, where CPU hardware manages this process dynamically during code execution.

8.1 Register Renaming Investigation

The CPU should be configured with the following parameters in both cases:

- Issue Width = 1
- Register Read Ports = 4
- Register Write ports = 4
- Integer ALU = 4
- Float ALU = 1
- Shift/Binary Unit = 1
- CacheMode = 0 (unified cache)

Test Cases

- **test8a.txt** *(RAW and WAW Hazards)*
- **test8b.txt** *(RAW and WAR Hazards)*
- **test8c.txt** *(WAW hazards)*

Manually renamed versions of same files

- **test8ax.txt** *(RAW) (WAW fixed by renaming)*
- **test8bx.txt** *(RAW) (WAR fixed by renaming)*
- **test8cx.txt** *(none) (WAW fixed by renaming)*

Tasks

ii. We will use **week8std.cpp** to generate results for all six test cases. This program performs all of the optimisations we have developed up to and including week7. Examine the results for each case and fill in the table below.

	Week8std.cpp (Stalls)						***Week8std.cpp (Renaming)***			
test case	**Total cycles**	**Speedup**	**Regs Used**	**Stalls used**		**test case**	**Total cycles**	**Speedup**	**Regs Used**	**Stalls used**
test8a						*test8ax*				
test8b						*test8bx*				
test8c						*test8cx*				

You should see that WAW and WAR hazards are removed in both sets of test cases, but that stalls were not needed to do so for the 'x' versions. However RAW hazards always need to be fixed by using stalls - renaming is not helpful.

In case 8b we also see that register renaming was performed for the last writeback to register R13 at instruction i=4, but when examined in the pipeline diagram it is actually unnecessary - this is an example where pessimistic optimisation played safe but was unnecessary in the final execution of the code.

8.2 An automated solution

Register renaming can be applied statically, as we did manually in 8.1. However we can also apply register renaming directly in the pipeline in a dynamic fashion. This would normally be performed by the CPU hardware during instruction issuing, and would require quite complex hardware to do so. We can however implement an algorithm which adjusts the pipeline in our emulator to rename registers where needed, and get a similar outcome.

i. Develop an algorithm which detects the WAW and WAR hazards in the pipeline and uses renaming to fix these. Your algorithm should act on the pipeline directly, detecting WAR and WAW cases and then renaming registers following on from that point where required.

Use **Week8ren.cpp** as a starting point and add your code in the **RenameRegisters(…)** function body provided in that file .

Helpful Hints:

1. You will need to generate new register names dynamically in your algorithm. There are ideal solutions, but more complex to implement. However a simple approach for our purposes is to start register renaming at R20 and just increment a counter for the register ID for each new renaming case performed. Ie R20, R21, R22, etc.

2. You might find it useful to disable the other register optimisations whilst developing and testing your renaming algorithm, but remember to re-enable them in your final solution.

8.3 A more efficient register hazard solution ?

Week8std.cpp implemented a solution for all hazards, but used the inefficient approach of inserting stalls. We also used manual renaming to achieve a pessimistic static optimisation to remove WAR and WAW hazards. Your solution to **Week8ren.cpp** uses renaming to solve the WAW and WAR hazards but still uses stalls to resolve RAW hazards and other issues such as structural hazards. Your solution may be more optimal than the pessimistic solution.

You should now be able to fill in a table as given below with the results you have generated, and see the benefits of each technique. Compare the speedups against the original serial execution, and count the number of unique registers used (e.g. if R3, R2 and R7 are used that is 3 registers).

	Week8std.cpp (Stalls)						***Week8ren.cpp (Renaming)***			
test case	**Total cycles**	**Speedup**	**Regs Used**	**Stalls used**		**test case**	**Total cycles**	**Speedup**	**Regs Used**	**Stalls used**
test8ax						*test8a*				
test8bx						*test8b*				
test8cx						*test8c*				

We should be able to see that dynamic renaming generally uses the same or fewer registers but also reduces the number of stalls required. Manual remaining is less efficient therefore, and creates a demand for larger register files (actually both static and dynamic renaming have this effect, but our simple static approach is more aggressive in its use of registers).

It is important to realise that by making the register file larger we may create additional costs that outweigh the benefits of register renaming alone. These include the hardware cost of having a larger register file and/or the effect of 'register pressure' and 'register spilling'. A more 'intelligent' algorithm might try to reuse register identities as they can be identified as no longer active or 'live', this would further reduce the need for more registers.Some notes on these additional topics may be found at https://en.wikipedia.org/wiki/Register_allocation

8.4 Solutions

Program Solutions **week8s1.cpp** will be released at the end of week 8. Answers for Part 8.2 and 8.3 will also be released as **labsheet.week.8.Table.pdf**